

KAIROX BROWSER

Master Product + Engineering Build Command

September 17, 2026

Contents

1	0. MASTER EXECUTION COMMAND	1
1.1	Command to the engineering AI	1
1.2	Non-negotiable rules	1
2	1. September 2026 Platform Baseline	1
2.1	1.1 Current Android generation	1
2.2	1.2 Current build stack	1
2.3	1.3 Minimum Android version policy	2
3	2. Product Definition	2
3.1	2.1 Product statement	2
3.2	2.2 User modes	2
3.2.1	Everyday mode	2
3.2.2	Developer mode	2
3.2.3	Privacy mode	3
3.2.4	Power mode	3
4	3. Architecture	3
4.1	3.1 Layer model	3
4.2	3.2 Dependency direction	4
4.3	3.3 Repository layout	4
5	4. Phase 0 - Repository Audit and Migration Map	5
6	5. Phase 1 - Toolchain and CI Foundation	5
6.1	5.1 Build configuration	5
6.2	5.2 Release artifact pipeline	6
6.3	5.3 Phase gate	6
7	6. Phase 2 - GeckoView Browser Engine	6
7.1	6.1 Engine contract	6
7.2	6.2 Engine lifecycle	6
7.3	6.3 Real-browser gate	6
8	7. Phase 3 - Profile Vault and Hard Isolation	7
8.1	7.1 Profile model	7
8.2	7.2 Profile creation	7
8.3	7.3 Profile clone	7
8.4	7.4 Temporary profile	7
8.5	7.5 Isolation test	8
9	8. Phase 4 - Browser Core	8
9.1	8.1 Toolbar	8
9.2	8.2 Tabs	8
9.3	8.3 Workspaces	8
9.4	8.4 History and bookmarks	8
10	9. Phase 5 - Privacy Center	9
10.1	9.1 Privacy policy object	9
10.2	9.2 Modes	9

10.2.1	Balanced	9
10.2.2	Strict	9
10.2.3	Developer	9
10.2.4	Temporary	9
10.3	9.3 Per-site controls	9
11	10. Phase 6 - Browser Identity / Presentation Lab	9
11.1	10.1 Presets	9
11.2	10.2 Important rule	10
11.3	10.3 Header Lab	10
12	11. Phase 7 - Network Lab	10
12.1	11.1 Routing model	10
12.2	11.2 Proxy entity	10
12.3	11.3 Trust versus health	10
13	12. Phase 8 - Dynamic Proxy Discovery and Health System	11
13.1	12.1 Supported sources	11
13.2	12.2 Discovery pipeline	11
13.3	12.3 Health state machine	11
13.4	12.4 Bounded concurrency	11
13.5	12.5 Secure behavior	11
14	13. Phase 9 - Network Diagnostics	11
15	14. Phase 10 - Developer Lab	12
15.1	14.1 Inspector	12
15.2	14.2 Console	12
15.3	14.3 Network inspector	12
15.4	14.4 Storage inspector	12
15.5	14.5 Capture	12
15.6	14.6 Source and metadata	13
16	15. Phase 11 - Page Extraction / Research Fetcher	13
17	16. Phase 12 - Downloads and Files	13
18	17. Phase 13 - AI Companion	14
18.1	Provider interface	14
18.2	Tasks	14
18.3	Data safety	14
19	18. Phase 14 - Automation Sandbox	14
20	19. Phase 15 - Data Model	14
20.1	Core tables	14
20.1.1	profiles	14
20.1.2	tabs	15
20.1.3	proxy_endpoints	15
20.1.4	site_permissions	15
20.1.5	downloads	15
20.1.6	AI providers	15
21	20. Phase 16 - Storage and Secrets	15
22	21. Phase 17 - Security Boundary Review	16
23	22. Phase 18 - UI/UX System	16
23.1	22.1 Design language	16
23.2	22.2 Bottom navigation	16
23.3	22.3 Home	16
23.4	22.4 Profile Vault	17
23.5	22.5 Network Lab	17
23.6	22.6 Developer Lab	17

24	23. Phase 19 - Accessibility and Internationalization	17
25	24. Phase 20 - Performance and Memory	18
25.1	Strategy	18
25.2	Performance matrix	18
26	25. Phase 21 - Crash Recovery	18
26.1	Engine crash	18
26.2	Process death	18
27	26. Phase 22 - Compatibility Matrix	18
28	27. Phase 23 - Android 17 Specific Gates	19
29	28. Phase 24 - Testing Strategy	19
29.1	Mandatory suites	19
29.1.1	Profile isolation	19
29.1.2	Network	19
29.1.3	Developer Lab	19
29.1.4	Security	19
29.1.5	Accessibility	19
30	29. Phase 25 - Exact Test Scripts	19
30.1	TEST_PROFILE_ISOLATION_001	19
30.2	TEST_NETWORK_001	20
30.3	TEST_DEVTOOLS_001	20
31	30. Phase 26 - CI / Release Pipeline	20
31.1	Pull request pipeline	20
31.2	Nightly pipeline	20
31.3	Release candidate pipeline	21
31.4	Artifact manifest	21
32	31. Phase 27 - Release Channels	21
32.0.1	Nightly	21
32.0.2	Beta	21
32.0.3	Stable	21
32.0.4	Emergency engine update	21
33	32. Phase 28 - Store and Distribution	21
34	33. Phase 29 - Privacy-Conscious Product Analytics	22
35	34. Phase 30 - Advanced Settings Catalog	22
35.1	Appearance	22
35.2	Tabs	22
35.3	Search	22
35.4	Privacy	22
35.5	Identity	22
35.6	Network	23
35.7	Developer	23
35.8	Downloads	23
35.9	AI	23
35.10	Experimental	23
36	35. Phase 31 - UI Interaction Rules	23
37	36. Phase 32 - Browser Presentation Verification	23
38	37. Phase 33 - Profile Resource Strategy for 50 Profiles	24
39	38. Phase 34 - Profile Export Format	24
40	39. Phase 35 - Browser Failure Containment	24

41	40. Phase 36 - Documentation the Agent Must Maintain	25
42	41. Phase 37 - Machine-Readable Phase Status	25
43	42. Phase 38 - Final Release Definition of Done	25
44	43. Phase 39 - Final AI Agent Command Block	26
45	44. Appendix A - September 2026 Official References	27
46	45. Appendix B - Do Not Ship Checklist	28
47	46. Final Product Identity	28

KAIROX

The Mobile Browser / Profile Vault / Developer Lab / Privacy Lab / Network Lab

MASTER PRODUCT + ENGINEERING BUILD COMMAND

September 17, 2026 baseline

This document is an execution specification for a senior engineering agent or development team. It is designed to be followed step by step, with real implementation, real testing, real browser-engine integration, real profile isolation, and release gates. It explicitly rejects mock-only functionality and stale platform assumptions.

1 0. MASTER EXECUTION COMMAND

1.1 Command to the engineering AI

You are the lead engineering agent for Kairox Browser. Build a production-grade Android browser, not a WebView wrapper, not a visual prototype, and not a collection of disconnected screens. Work directly in the repository. Preserve working functionality while upgrading the system. Every exposed control must be wired to real behavior or be explicitly classified as experimental and hidden from normal users.

The date baseline is September 17, 2026. Do not use an older Android platform plan merely because it is familiar. Android 17 is API level 37. Google Play requires new apps and updates to target Android 16/API 36 or higher as of August 31, 2026. Kairox targets API 37.

Primary engine: GeckoView behind an internal BrowserEngine abstraction. UI: Jetpack Compose + Material 3. Toolchain: JDK 17, AGP 9.4.0, Gradle 9.6.0, Kotlin Gradle Plugin 2.2.10. Pin release dependencies; do not use floating versions.

Product scope: full mobile browsing, up to 50 isolated profiles, profile cloning/import/export/temporary profiles, per-profile cookies/storage/history/downloads/permissions, tab groups, workspaces, developer tooling, privacy controls, network routing, proxy discovery/health validation, browser-presentation controls, local-first AI, page extraction/research tooling, downloads, crash recovery, memory-aware tab management, security controls, CI/CD, release signing, and Play-ready distribution.

Work in phases. At the end of every phase: build, run tests, install, execute a real smoke flow, inspect logs for leaks, fix defects, write STATUS.md, and commit. Never claim completion because the screen looks correct. Completion means a real capability works and has a test.

1.2 Non-negotiable rules

1. Do not replace live browser functionality with mock values.
2. Do not silently downgrade to a WebView-only architecture.
3. Do not share cookies, site storage, permissions, history, downloads, or session state between profiles unless the user explicitly exports/imports data.
4. Do not store passwords, cookies, proxy credentials, API keys, or authorization headers in plaintext.
5. Do not perform blocking network operations on the UI thread.
6. Do not expose unknown public proxies as trusted infrastructure.
7. Do not implement CAPTCHA solving, rate-limit evasion, access-control bypass, platform-ban evasion, credential interception, or claims of guaranteed undetectability.
8. Do not ship an obsolete browser engine simply to preserve support for an old Android version.
9. Do not make account registration mandatory for normal browsing.
10. Do not declare release readiness until the final acceptance gates pass.

2 1. September 2026 Platform Baseline

2.1 1.1 Current Android generation

As of September 17, 2026:

- Android 17 is API level 37.
- Google Play's current requirement says new apps and updates must target Android 16/API 36 or higher starting August 31, 2026.
- Kairox should target API 37 rather than merely targeting the minimum Play requirement.
- Android 17 introduces behavior changes that include RAM-based app memory limits; memory management therefore belongs in the core architecture.

2.2 1.2 Current build stack

Component	Decision	Gate
Compile SDK	API 37	Clean build on CI and local workstation
Target SDK	API 37	Android 17 test matrix passes
AGP	9.4.0	Use the documented Gradle/JDK compatibility
Gradle	9.6.0	Wrapper committed and pinned

Component	Decision	Gate
JDK	17	JAVA_HOME and CI must agree
Kotlin Gradle Plugin	2.2.10	Pinned in version catalog/build logic
UI	Jetpack Compose + Material 3	Adaptive, accessible, edge-to-edge
Engine	GeckoView first	Engine version pinned per release train
Data	Room + DataStore	Migrations tested
Secrets	Android Keystore-backed encryption	No plaintext credentials
Background work	WorkManager/coroutines	Bounded and cancellable

2.3 1.3 Minimum Android version policy

The requested target range includes Android 7.0 through Android 17. Treat that as a compatibility objective, not a marketing claim.

Set the minimum supported Android version to API 24 only if the chosen current browser engine, graphics path, storage model, TLS behavior, permissions, and all critical Kairox functionality pass testing on API 24. If the current engine's secure support floor rises, document the exact failure and keep a separate compatibility strategy rather than embedding an obsolete insecure engine.

The principle is simple: an installable APK is not proof of browser compatibility.

3 2. Product Definition

3.1 2.1 Product statement

Kairox = one browser, many isolated worlds.

The browser must feel easy for a normal user and powerful for a developer. Complexity is organized behind dedicated surfaces instead of being forced into the main browsing screen.

3.2 2.2 User modes

3.2.1 Everyday mode

- Search and address bar
- Tabs and tab groups
- Bookmarks and history
- Downloads
- Reader mode where supported
- Dark/light theme
- Share
- Find in page
- Translation where supported
- PWA/add-to-home-screen support where supported
- Private/temporary browsing
- Site permissions

3.2.2 Developer mode

- Inspector
- Console
- Network panel
- Storage inspector
- Header viewer
- Source viewer
- Screenshot/full-page capture
- Page metadata extractor
- JSON/CSV/Markdown export
- API scratchpad
- Automation sandbox
- Performance diagnostics

3.2.3 Privacy mode

- Tracking protection
- Cookie/storage policies
- HTTPS-first
- WebRTC policy
- Fingerprint-protection configuration
- Per-site exceptions
- Permission dashboard
- DNS controls
- Network route controls

3.2.4 Power mode

- 50 profiles
- Profile cloning
- Temporary profiles
- Workspaces
- Proxy pools
- Network health checks
- Presentation presets
- Research jobs
- Local AI
- Advanced automation

4 3. Architecture

4.1 3.1 Layer model

Kairox App Shell

```
|
+-- UI / Navigation / Adaptive Windows
|
+-- Browser Domain
|   +-- BrowserEngine contract
|   +-- Tabs
|   +-- Sessions
|   +-- Downloads
|
+-- Gecko Engine Adapter
|   +-- GeckoRuntime
|   +-- GeckoSession
|   +-- Content Blocking
|   +-- Permissions
|   +-- WebExtensions bridge
|
+-- Profile Core
|   +-- Registry
|   +-- Isolation
|   +-- Import / Export
|   +-- Temporary profiles
|
+-- Privacy Core
|   +-- Tracking policy
|   +-- Site permissions
|   +-- Storage policy
|   +-- Fingerprint policy
|
+-- Network Core
|   +-- Route manager
|   +-- Proxy registry
|   +-- Health checker
|   +-- DNS policy
```

```

|      +--- Diagnostics
|
+--- Developer Lab
|      +--- Inspector
|      +--- Console
|      +--- Network
|      +--- Storage
|      +--- Headers
|      +--- Extraction
|      +--- Automation
|
+--- Data / Secrets / Background Work

```

4.2 3.2 Dependency direction

UI depends on domain contracts. Domain contracts do not depend on UI. Browser business logic does not know Gecko-specific classes. Gecko implementation satisfies the engine interface. Profile logic controls profile identity and data policy; the engine adapter is responsible for translating those requirements into engine APIs.

4.3 3.3 Repository layout

```

kairox/
  app/
  core/
    common/
    model/
    logging/
    result/
  browser/
    browser-api/
    browser-state/
    navigation/
    tabs/
    downloads/
  engine/
    engine-api/
    engine-gecko/
  profiles/
    profile-api/
    profile-data/
    profile-storage/
    profile-isolation/
    profile-backup/
  privacy/
    privacy-api/
    tracking-protection/
    site-permissions/
    storage-policy/
    fingerprint-policy/
  network/
    network-api/
    route-manager/
    proxy-registry/
    proxy-health/
    dns-policy/
    diagnostics/
  developer-lab/
    devtools-api/
    inspector/
    console/
    network-inspector/
    storage-inspector/
    headers-lab/

```

```
    page-extractor/
    automation/
ai/
  ai-api/
  local-provider/
  remote-provider/
ui/
  design-system/
  home/
  browser/
  tabs/
  profiles/
  privacy/
  network/
  developer/
  settings/
testing/
  unit/
  instrumentation/
  profile-isolation/
  browser-compat/
  security/
  performance/
build-logic/
  convention-plugins/
  dependency-catalog/
```

5 4. Phase 0 - Repository Audit and Migration Map

Before writing feature code:

1. Read the entire existing project structure.
2. Record applicationId and package names.
3. Record current AGP, Gradle, Kotlin, compileSdk, targetSdk, minSdk.
4. Record the current browser engine and every browser-related dependency.
5. Record data stores, databases, preferences, file paths, background workers and services.
6. Record existing tests and their current pass/fail status.
7. Record signing configuration without exposing secrets.
8. Record current release artifacts.
9. Identify dead code and duplicated navigation/state layers.
10. Produce docs/MIGRATION_PLAN.md.

The agent must not delete old functionality until its purpose is understood and either migrated or formally retired.

Gate: the repository builds before major changes, or the exact pre-existing build failure is documented in STATUS.md.

6 5. Phase 1 - Toolchain and CI Foundation

6.1 5.1 Build configuration

Implement:

- Gradle wrapper pinned to 9.6.0.
- AGP 9.4.0.
- JDK 17 toolchain.
- Kotlin Gradle Plugin 2.2.10.
- compileSdk 37.
- targetSdk 37.
- dependency version catalog.
- dependency locking.
- explicit repositories only.
- CI build using the same JDK and Gradle wrapper.

6.2 5.2 Release artifact pipeline

Generate:

- Debug APK
- Release APK for direct testing where appropriate
- Release AAB
- R8/ProGuard mapping
- SBOM
- dependency lockfile
- version manifest
- checksums

Do not put API keys in Gradle files or CI logs.

6.3 5.3 Phase gate

A clean checkout on CI must be able to:

```
./gradlew clean test lint assembleDebug
```

and produce a valid installable artifact before browser functionality is added.

7 6. Phase 2 - GeckoView Browser Engine

Use GeckoView as the first engine implementation and keep it behind `BrowserEngine`.

7.1 6.1 Engine contract

The contract must expose at minimum:

```
createRuntime()
createSession(profileContext)
loadUrl(session, url)
stop(session)
reload(session)
goBack(session)
goForward(session)
currentUrl(session)
currentTitle(session)
observeNavigation(session)
observeContentState(session)
setPermissionPolicy(session, policy)
setContentBlockingPolicy(session, policy)
saveAsPdf(session)
captureFullPage(session)
destroySession(session)
```

7.2 6.2 Engine lifecycle

- One application-level engine runtime where supported and safe.
- Profile-aware session creation.
- Warm-up support where available.
- Crash callbacks.
- Low-memory hooks.
- Session restore state outside the engine.
- Version reporting in the About screen.

7.3 6.3 Real-browser gate

The agent must load a real HTTPS website on a physical Android device and verify:

- navigation
- redirect
- JavaScript

- form entry
- back/forward
- page reload
- file download
- file upload
- permissions
- screenshot

No mock page counts toward this gate.

8 7. Phase 3 - Profile Vault and Hard Isolation

8.1 7.1 Profile model

A profile is a first-class environment, not a visual label.

```
ProfileId
|
+-- Engine/browser storage context
+-- Cookies
+-- localStorage
+-- IndexedDB
+-- Cache
+-- History
+-- Bookmarks
+-- Downloads policy
+-- Site permissions
+-- Privacy policy
+-- Network route policy
+-- Presentation policy
+-- Workspace state
```

The app supports up to 50 user-created profiles.

8.2 7.2 Profile creation

Fields:

- Name
- Icon
- Accent
- Starting URL
- Search engine
- Privacy mode
- Network route
- Presentation preset
- Persistent vs temporary

Advanced configuration is optional during creation.

8.3 7.3 Profile clone

Clone can transfer:

- bookmarks
- settings
- workspace structure
- selected extensions

Clone must not copy login sessions or cookies unless the user explicitly chooses them.

8.4 7.4 Temporary profile

A temporary profile is designed to be destroyed on close. The UI must state exactly what is removed.

8.5 7.5 Isolation test

Create Profile A and Profile B. Store different cookie and localStorage values. Grant different permissions. Download different files. Restart the app. Verify that every profile retains only its own data. Delete A and verify B remains unchanged.

Pass condition: zero unintended cross-profile leakage.

9 8. Phase 4 - Browser Core

9.1 8.1 Toolbar

Primary mobile toolbar:

Back Forward [secure origin / search / URL] Tabs Menu

The address bar must support:

- URL detection
- search detection
- paste and go
- copy URL
- open in new profile
- QR/share
- page actions

9.2 8.2 Tabs

Implement:

- tab creation
- close
- duplicate
- pin
- move
- tab groups
- restore last session
- private/temporary tabs
- profile-bound tabs

9.3 8.3 Workspaces

A workspace contains a profile-scoped collection of tabs/groups and optionally a startup state.

Example:

Development

GitHub
Cloudflare
Render
Neon

Research

Docs
Search
Notes

Personal

Mail
Shopping

9.4 8.4 History and bookmarks

All records are profile scoped. Search and indexing must remain responsive with large datasets.

10 9. Phase 5 - Privacy Center

10.1 9.1 Privacy policy object

```
PrivacyPolicy
  trackingProtectionMode
  thirdPartyCookiePolicy
  storagePartitioningPolicy
  httpsFirst
  safeBrowsing
  webRtcPolicy
  fingerprintProtectionMode
  referrerPolicy
  permissionsPolicy
  siteExceptions[]
```

10.2 9.2 Modes

10.2.1 Balanced

Mainstream compatibility with strong baseline tracking protection.

10.2.2 Strict

Stronger storage and tracker restrictions with a compatibility warning.

10.2.3 Developer

Shows more diagnostic information and makes any privacy weakening explicit.

10.2.4 Temporary

For disposable browsing sessions.

10.3 9.3 Per-site controls

For the current origin show:

- tracking protection
- cookies
- storage
- location
- camera
- microphone
- notifications
- popups
- autoplay where supported
- JavaScript policy if exposed by the engine

11 10. Phase 6 - Browser Identity / Presentation Lab

The user asked for user-agent controls and desktop-browser presentation. Implement them honestly.

11.1 10.1 Presets

- Standard engine-native
- Android phone
- Android tablet
- Desktop-oriented presentation
- Custom supported fields
- Per-profile presentation policy
- Per-session override

11.2 10.2 Important rule

Changing one user-agent string does not change every browser characteristic. The UI must identify:

- values Kairox controls
- values the engine controls
- values Android controls
- unsupported fields

Do not market this as “undetectable”. Treat it as browser compatibility/testing/privacy configuration.

11.3 10.3 Header Lab

Allow developers to inspect request/response headers exposed through supported engine APIs.

Display the distinction between:

Engine generated

Application configured

Site returned

12 11. Phase 7 - Network Lab

12.1 11.1 Routing model

RoutePolicy

DIRECT

SYSTEM_VPN

MANUAL_PROXY

USER_PROXY_POOL

FAILOVER_POLICY

The current route must always be visible.

12.2 11.2 Proxy entity

ProxyEndpoint

id

protocol

host

port

credentialRef

countryHint

sourceId

trustState

healthState

latencyMs

successRate

consecutiveFailures

lastCheckedAt

12.3 11.3 Trust versus health

These are separate properties.

A proxy can be:

- technically reachable but untrusted
- user supplied and usable
- verified by the user’s own infrastructure
- failed
- blocked by policy

A high health score does not prove privacy or safety.

13 12. Phase 8 - Dynamic Proxy Discovery and Health System

The requested dynamic proxy functionality can be implemented as a controlled ingestion and health subsystem.

13.1 12.1 Supported sources

- User-supplied text files
- CSV feeds
- JSON feeds
- Provider APIs configured by the user
- Local proxy services
- Future source adapters through a stable interface

13.2 12.2 Discovery pipeline

Source

```
-> fetch
-> parse
-> normalize
-> deduplicate
-> syntax validate
-> connectivity test
-> protocol/TLS/HTTP sanity check
-> latency measurement
-> success-rate measurement
-> trust labeling
-> cache
-> candidate pool
```

13.3 12.3 Health state machine

```
NEW -> TESTING -> HEALTHY
NEW -> TESTING -> FAILED
HEALTHY -> DEGRADED -> RETESTING
FAILED -> RETESTING
DEGRADED -> BLOCKED when the configured policy requires it
```

13.4 12.4 Bounded concurrency

Never create unlimited proxy-check tasks. Use a bounded coroutine pool, cancellation, timeouts and exponential backoff.

13.5 12.5 Secure behavior

Do not route passwords, cookies or payment credentials through unknown public proxies by default. Display a strong warning for UNVERIFIED endpoints and require explicit user selection.

Do not build CAPTCHA bypass, rate-limit evasion or anti-ban routing.

14 13. Phase 9 - Network Diagnostics

The Network Lab should show a route graph:

```
Device
|
Wi-Fi / Mobile
|
Kairox
|
Route / Proxy / VPN
|
Internet
```

Diagnostics:

-
- DNS timing
 - TCP/connect timing where accessible
 - TLS timing where accessible
 - HTTP timing
 - route status
 - endpoint latency
 - failure reason
 - last successful check

Exports must redact proxy credentials.

15 14. Phase 10 - Developer Lab

15.1 14.1 Inspector

Tap a page element and show, where engine APIs permit:

- element type
- selector/path
- attributes
- text
- related style metadata
- parent/child structure

15.2 14.2 Console

Capture real page console events:

- log
- info
- warning
- error
- timestamps

Support filtering and search.

15.3 14.3 Network inspector

Show real session traffic that the engine exposes:

- request method
- URL
- status
- timing
- transfer size where available
- request/response headers where available
- content type

15.4 14.4 Storage inspector

Show profile-scoped site data:

- cookies
- localStorage
- IndexedDB summaries where supported
- storage usage

15.5 14.5 Capture

- viewport screenshot
- full-page screenshot
- share/save/export

15.6 14.6 Source and metadata

- page source
- page title
- canonical URL
- metadata
- links
- structured content where available

16 15. Phase 11 - Page Extraction / Research Fetcher

The browser can provide a transparent research fetcher for content the user is permitted to access.

Pipeline:

URL

```
-> access policy check
-> rate limiter
-> route selection
-> fetch
-> content-type validation
-> parser
-> readability/text extraction
-> metadata extraction
-> structured result
-> export
```

Export formats:

- JSON
- CSV
- Markdown
- HTML
- TXT

Controls:

- per-domain concurrency limit
- request delay
- maximum response size
- redirect cap
- cancellation
- retry/backoff
- job history

Do not build it as an uncontrolled high-volume crawler.

17 16. Phase 12 - Downloads and Files

Download manager requirements:

- per-profile default directory
- progress
- pause/resume where supported
- retry
- open
- share
- rename
- delete
- duplicate-name policy
- dangerous-file warning
- download history

Profile export/import must use Android's document picker and validate archive paths to prevent traversal attacks.

18 17. Phase 13 - AI Companion

AI is optional. Browsing must work with AI disabled.

18.1 Provider interface

```
AiProvider  
  LOCAL_OLLAMA  
  USER_CONFIGURED_HTTP_ENDPOINT  
  FUTURE_CLOUD_PROVIDER  
  DISABLED
```

18.2 Tasks

- explain page
- summarize page
- explain console error
- inspect screenshot
- generate code snippet
- transform selected text
- research assistant

18.3 Data safety

Before a cloud request, show what is being sent. Strip cookies, authorization headers, passwords, proxy credentials and private keys. Local-only mode must be available.

19 18. Phase 14 - Automation Sandbox

Automation must be explicit and observable.

Example workflow:

```
Open URL  
Wait for condition  
Select element  
Click / Type / Scroll  
Capture evidence  
Assert condition  
Export result
```

Required controls:

- start
- pause
- stop
- kill
- dry-run
- logs
- current step
- error details

The automation engine must not run hidden background workflows.

20 19. Phase 15 - Data Model

20.1 Core tables

20.1.1 profiles

```
id  
name  
icon  
accent  
created_at
```

last_used_at
temporary
startup_url
search_engine_id
privacy_policy_id
network_policy_id
presentation_policy_id

20.1.2 tabs

id
profile_id
workspace_id
url
title
favicon_uri
pinned
private
last_active_at

20.1.3 proxy_endpoints

id
protocol
host
port
credential_ref
source_id
trust_state
health_state
latency_ms
success_rate
last_checked_at

20.1.4 site_permissions

profile_id
origin
permission_type
decision
expires_at

20.1.5 downloads

profile_id
source_url
local_uri
status
bytes_downloaded
total_bytes
checksum

20.1.6 AI providers

id
type
base_url
credential_ref
enabled

21 20. Phase 16 - Storage and Secrets

Use:

- Room for structured app data
- DataStore for lightweight preferences
- engine-owned storage for browser content
- Android Keystore-backed secret storage for credentials
- authenticated encryption for profile exports

Never log:

- Cookie
- Set-Cookie
- Authorization
- Proxy-Authorization
- API keys
- passwords
- session tokens

Build a reusable secret-redaction function and test it against structured and unstructured logs.

22 21. Phase 17 - Security Boundary Review

Before feature freeze, review:

1. Web-to-native bridges.
2. WebExtensions permissions.
3. Imported archive parsing.
4. URL and custom-scheme handling.
5. File path normalization.
6. Download handling.
7. Secret storage.
8. AI context serialization.
9. Proxy credential handling.
10. Crash-report redaction.
11. Debug endpoint exposure.
12. Local network access.

Any developer-only endpoint must be inaccessible from ordinary web content.

23 22. Phase 18 - UI/UX System

23.1 22.1 Design language

Use a premium but restrained visual system:

- dark graphite base
- warm white light mode
- one strong accent
- soft depth instead of heavy blur
- 120-220 ms interaction motion
- no animation that blocks input
- no clutter on the home screen

23.2 22.2 Bottom navigation

Home Tabs Profiles Lab Menu

23.3 22.3 Home

KAIROX

[Search or enter address]

Continue browsing

[cards]

Workspaces

[Development] [Research] [Personal]

Power shortcuts

[Inspector] [Network] [Privacy] [Capture]

Privacy: Balanced Route: Direct Engine: Gecko

23.4 22.4 Profile Vault

Profiles +

Search profiles...

Personal	8 tabs
Development	12 tabs
Client A	3 tabs
Research	4 tabs
Temporary	0 tabs

Long press actions:

- open
- clone
- export
- clear
- delete
- archive

23.5 22.5 Network Lab

Show route, proxy pool status, current endpoint, trust state, latency and diagnostics.

23.6 22.6 Developer Lab

Use tool tiles:

- Inspector
- Console
- Network
- Storage
- Headers
- Source
- Capture
- Extract
- API

Every tile must open a real tool.

24 23. Phase 19 - Accessibility and Internationalization

Required:

- TalkBack labels
- keyboard focus where applicable
- large text support
- no information conveyed by color alone
- RTL support
- locale-aware formatting
- resource-based strings
- adequate touch targets
- adaptive layouts for phones, tablets, foldables and resizable windows

Developer panels must remain usable at large font sizes.

25 24. Phase 20 - Performance and Memory

Android 17 requires explicit attention to memory limits. Kairox must not create 50 fully live heavyweight engine sessions just because 50 profiles exist.

25.1 Strategy

- Keep profile metadata light.
- Activate engine sessions only as needed.
- Persist tabs/workspaces separately from live engine objects.
- Suspend inactive tabs under memory pressure.
- Recreate sessions from recoverable state.
- Bound proxy health checks.
- Stream downloads directly to disk.
- Keep large history queries paged/indexed.

25.2 Performance matrix

Test:

- cold start
- warm start
- 5/10/25/50 tabs
- 1/5/10/25/50 profiles
- long history
- large downloads
- multiple proxy checks
- low-memory conditions
- process death and recovery

Record p50/p95 internally for startup and common navigation flows.

26 25. Phase 21 - Crash Recovery

26.1 Engine crash

1. Persist recoverable navigation state.
2. Record sanitized crash metadata.
3. Mark the affected tab RECOVERABLE.
4. Recreate the engine session where safe.
5. Restore the URL/history state.
6. Keep unrelated profiles alive.
7. Offer Close or Recover.

26.2 Process death

The browser shell must appear before waiting for optional network diagnostics. Restore active profile, workspace and recoverable tabs without making startup depend on the network.

27 26. Phase 22 - Compatibility Matrix

Android	Required validation
7 / API 24	install, engine, navigation, isolation, downloads, permissions, crash recovery
8-10	above + background/permission transitions
11-13	scoped storage, notifications, media, permissions
14-15	edge-to-edge and current permission behavior
16 / API 36	Play baseline and target behavior
17 / API 37	current target behavior, memory limits, local-network/security changes

The release dashboard must distinguish:

- verified supported
- installed but not fully tested
- unsupported by current engine
- blocked by platform limitation

28 27. Phase 23 - Android 17 Specific Gates

Before release on API 37:

- run instrumentation suite
- test memory pressure
- test resizable windows
- test local network behavior used by developer tools
- test loopback assumptions
- test downloads
- test notifications
- test file picker
- test permissions
- test background/foreground transitions
- test crash recovery

Do not assume Android 17 behaves like Android 16.

29 28. Phase 24 - Testing Strategy

Every feature needs:

1. unit test where logic is deterministic
2. repository test for persistence
3. instrumentation test for Android integration
4. browser-engine integration test where needed
5. end-to-end smoke flow

29.1 Mandatory suites

29.1.1 Profile isolation

Prove cookie/storage/history/permission/download separation.

29.1.2 Network

Prove timeout, retry, cancellation, route switch and redaction.

29.1.3 Developer Lab

Prove console, network, storage and capture reflect live page data.

29.1.4 Security

Prove archive traversal defenses, secret redaction, invalid URL handling and native-bridge boundaries.

29.1.5 Accessibility

Prove TalkBack labels, text scaling, focus order and non-color states.

30 29. Phase 25 - Exact Test Scripts

30.1 TEST_PROFILE_ISOLATION_001

1. Install clean build.
2. Create Profile A and Profile B.
3. Open controlled local test site in A.

-
4. Write cookie A=alpha.
 5. Write localStorage A=alpha.
 6. Grant permission in A.
 7. Open same site in B.
 8. Assert A values are absent.
 9. Write B=beta values.
 10. Restart app.
 11. Assert both profiles retain only their own state.
 12. Delete A.
 13. Assert B is unchanged.

PASS = zero leakage.

30.2 TEST_NETWORK_001

1. Add a user-supplied test endpoint.
2. Mark USER_SUPPLIED.
3. Run health check.
4. Record latency and success.
5. Route an allowed test request.
6. Show route diagnostics.
7. Force timeout.
8. Verify bounded retry.
9. Verify endpoint state changes.
10. Switch to DIRECT.
11. Verify the next request uses DIRECT.
12. Export diagnostics with credentials removed.

PASS = browser remains responsive and route state matches actual behavior.

30.3 TEST_DEVTOOLS_001

1. Open controlled test page.
2. Emit console.log/warn/error.
3. Create GET and POST requests.
4. Inspect live request list.
5. Open a request.
6. Inspect status/timing/headers exposed by engine.
7. Inspect storage.
8. Capture full-page screenshot.
9. Extract page text/metadata.
10. Close Lab.
11. Reload page.
12. Verify normal browsing remains intact.

31 30. Phase 26 - CI / Release Pipeline

31.1 Pull request pipeline

```
format/lint
-> unit tests
-> static analysis
-> dependency scan
-> instrumentation smoke
-> debug build
```

31.2 Nightly pipeline

```
browser compatibility
-> profile isolation
-> network
-> security
-> memory/performance
-> device matrix
```

31.3 Release candidate pipeline

```
full matrix
-> SBOM
-> signed artifacts
-> install/launch verification
-> Play pre-check
-> privacy/data-safety review
-> release notes
-> tag
```

31.4 Artifact manifest

Every release must include:

- version
- commit SHA
- engine version
- target/compile SDK
- toolchain versions
- dependency lock state
- signing identity reference
- test summary
- known limitations

32 31. Phase 27 - Release Channels

32.0.1 Nightly

Latest engine/product integration. Not guaranteed stable.

32.0.2 Beta

Feature-complete candidate with release-gate testing.

32.0.3 Stable

Signed production build after all gates.

32.0.4 Emergency engine update

Fast path for critical browser-engine security updates.

Do not keep a materially stale engine merely to avoid a release.

33 32. Phase 28 - Store and Distribution

Primary: Google Play AAB.

Secondary: official signed APK distribution under the Kairox-controlled domain.

Required:

- Play App Signing
- data-safety disclosure
- privacy policy
- security contact
- staged rollout
- rollback plan
- versioned release notes
- clear engine version disclosure

34 33. Phase 29 - Privacy-Conscious Product Analytics

Collect only what is required to improve the product.

Acceptable default signals:

- crash count
- anonymous build/channel
- engine version
- feature-toggle state
- performance timing without page content

Do not collect by default:

- full URLs
- browsing history
- search queries
- cookies
- page content
- proxy credentials

A privacy browser must not secretly create another history database.

35 34. Phase 30 - Advanced Settings Catalog

35.1 Appearance

- theme
- toolbar position
- compact mode
- animation intensity
- font scale

35.2 Tabs

- tab layout
- restore behavior
- tab grouping
- auto-suspend policy
- new-tab behavior

35.3 Search

- engine
- suggestions
- private search behavior

35.4 Privacy

- tracking protection
- cookies
- storage
- HTTPS-first
- WebRTC
- permissions
- fingerprint protection

35.5 Identity

- presentation presets
- per-profile policy
- supported custom fields

35.6 Network

- route
- proxy
- DNS
- health checks
- failover
- diagnostics

35.7 Developer

- console retention
- network capture
- source viewer
- extractor settings

35.8 Downloads

- location
- notifications
- duplicate-name policy
- file prompts

35.9 AI

- provider
- local/remote mode
- data review
- disable

35.10 Experimental

All experimental features must have a description, reset action and clear warning.

36 35. Phase 31 - UI Interaction Rules

Gesture/action	Required result
Long press profile	open / clone / export / clear / delete
Long press link	open / new tab / new profile / copy / share / inspect
Swipe tab	close / save / move according to chosen gesture map
Tap privacy icon	current-site privacy panel
Tap network indicator	current route + diagnostics
Tap developer icon	last-used developer tool
Back	browser history first, then tab/workspace navigation
Pull to refresh	reload without losing profile state

The app must never make users guess which profile they are currently in.

37 36. Phase 32 - Browser Presentation Verification

Create an internal page called `Kairox Identity`.

Display:

Current profile

Engine

Presentation preset

Public network address as observed by the selected diagnostic service

DNS summary where available

User agent

Client hints where exposed

WebRTC state

Cookies policy
Tracking protection
Route
Proxy trust state

The diagnostic screen describes what Kairox configured; it does not promise that a third-party fingerprint tester will see a perfect or identical identity.

38 37. Phase 33 - Profile Resource Strategy for 50 Profiles

The app must not keep 50 heavy sessions permanently active.

Recommended policy:

Profile metadata: always available
Profile storage: persistent
Live engine session: activate on demand
Inactive profile: persisted, engine released where safe
Active tabs: memory-aware
Workspace: serialized state

Stress test:

- create 50 profiles
- add metadata and bookmarks
- switch through all profiles
- open selected profiles concurrently where architecture permits
- measure memory
- trigger low-memory behavior
- recover without data loss

39 38. Phase 34 - Profile Export Format

Use a versioned encrypted package.

KXPROFILE

manifest.json
profile-settings.json
bookmarks.json
workspace.json
selected-history.json
selected-download-metadata.json
optional-extension-data/
integrity.json

Import must:

1. validate file type
2. validate version
3. validate integrity
4. validate archive paths
5. create a new profile ID
6. import only selected data
7. never overwrite an existing profile silently

40 39. Phase 35 - Browser Failure Containment

If a webpage or engine session fails:

- the app shell survives
- other profiles survive
- other tabs survive where possible
- the affected tab enters RECOVERABLE state
- logs are sanitized

-
- recovery is user-visible

The highest-priority reliability goal is to prevent one page from becoming a whole-app crash.

41 40. Phase 36 - Documentation the Agent Must Maintain

The repository must contain:

```
docs/  
  ARCHITECTURE.md  
  MIGRATION_PLAN.md  
  PROFILE_ISOLATION.md  
  NETWORK_MODEL.md  
  PRIVACY_MODEL.md  
  DEVELOPER_LAB.md  
  AI_MODEL.md  
  SECURITY_MODEL.md  
  TEST_PLAN.md  
  RELEASE_PROCESS.md  
  COMPATIBILITY.md  
  STATUS.md
```

Every phase updates relevant documentation.

42 41. Phase 37 - Machine-Readable Phase Status

Create STATUS.md with:

```
# KAIROX PHASE STATUS
```

```
Phase: 00
```

```
Date: 2026-09-17
```

```
Build: 0.x
```

```
Implemented:
```

```
- ...
```

```
Verified:
```

```
- ...
```

```
Tests:
```

```
- total: ...
```

```
- passed: ...
```

```
- failed: ...
```

```
Known limitations:
```

```
- ...
```

```
Security notes:
```

```
- ...
```

```
Performance notes:
```

```
- ...
```

```
Next phase:
```

```
- ...
```

```
Release blockers:
```

```
- ...
```

43 42. Phase 38 - Final Release Definition of Done

Kairox is release-ready only when all statements below are true.

- Clean CI build succeeds.
- Dependency versions are pinned.
- API 37 build passes.
- Android 17 tests pass.
- Profile isolation suite passes with zero leakage.
- Basic browsing works on real devices.
- Tab/session restore works.
- Downloads work.
- Site permissions work.
- Developer Lab shows live data.
- Network Lab handles timeouts without freezing UI.
- Proxy health distinguishes trust from reachability.
- Secrets never appear in logs.
- Imported archives are path-safe.
- AI sanitization works.
- Performance and memory limits are documented.
- Accessibility baseline passes.
- Privacy/data-safety declarations match the implementation.
- Signed artifacts install and launch.
- Browser engine security update process exists.
- No placeholder feature is presented as production-ready.

44 43. Phase 39 - Final AI Agent Command Block

Copy the following as the top-level instruction for the implementation agent:

YOU ARE THE LEAD ENGINEERING AGENT FOR KAIROX BROWSER.

You must build the product, not merely design it.

DATE BASELINE: 2026-09-17.

PRIMARY PLATFORM:

Android 17 / API 37.

Google Play minimum target for new apps/updates: API 36 or higher.

Kairox target: API 37.

TOOLCHAIN:

JDK 17.

AGP 9.4.0.

Gradle 9.6.0.

Kotlin Gradle Plugin 2.2.10.

ENGINE:

GeckoView behind BrowserEngine abstraction.

PRODUCT REQUIREMENTS:

- Full mobile browser.
- Up to 50 genuinely isolated profiles.
- Separate cookies, storage, history, permissions and downloads.
- Clone, export, import and temporary profiles.
- Tabs, tab groups, workspaces and session restore.
- Privacy Center.
- Network Lab.
- User-supplied and authorized proxy discovery/health system.
- Current route diagnostics.
- Browser presentation controls with truthful scope.
- Developer Lab with real session data.
- Page extraction/research jobs with rate limits and transparent status.
- Local-first AI provider architecture.
- Controlled automation.
- Download manager.

-
- Crash recovery.
 - Memory-aware resource management.
 - CI, security, SBOM, signing and Play-ready release process.

EXECUTION ORDER:

0. Audit repository.
1. Lock toolchain and CI.
2. Integrate GeckoView.
3. Build profile isolation.
4. Build browser shell.
5. Build privacy layer.
6. Build identity/presentation controls.
7. Build Network Lab.
8. Build Developer Lab.
9. Build research/extraction.
10. Build AI.
11. Build automation.
12. Harden storage/secrets.
13. Run performance and compatibility matrix.
14. Run security review.
15. Build signed release artifacts.

AFTER EACH PHASE:

- compile
- test
- install
- execute real smoke flow
- inspect logs
- fix defects
- update STATUS.md
- commit

NEVER:

- fake a feature with static demo data
- silently downgrade to WebView-only
- share profile state accidentally
- log secrets
- block the UI on network work
- treat unknown proxies as trusted
- implement CAPTCHA or rate-limit bypass
- claim guaranteed undetectability
- use an obsolete engine merely to keep an old Android claim

FINAL RULE:

Do not say "complete" until the Release Definition of Done passes.

If a requirement cannot be supported safely or technically on a particular Android version, document the exact constraint and implement the safest supported strategy instead of pretending.

45 44. Appendix A - September 2026 Official References

1. Google Play target API requirements: <https://developer.android.com/google/play/requirements/target-sdk>
2. Android 17 enterprise/API overview: <https://developer.android.com/work/versions/android-17>
3. Android 17 behavior changes: <https://developer.android.com/about/versions/17/behavior-changes-all>
4. Android Gradle Plugin 9.4.0 release notes: <https://developer.android.com/build/releases/agp-9-4-0-release-notes>
5. GeckoView documentation: <https://firefox-source-docs.mozilla.org/mobile/android/geckoview/index.html>
6. GeckoView quick start: <https://firefox-source-docs.mozilla.org/mobile/android/geckoview/consumer/geckoview-quick-start.html>
7. GeckoView API changelog: <https://mozilla.github.io/geckoview/javadoc/mozilla-central/org/mozilla/geckoview/doc-files/CHANGELOG>
8. GeckoView WebExtensions documentation: <https://firefox-source-docs.mozilla.org/mobile/android/geckoview/consumer/extensions.html>

Re-check these sources before every release because Android, AGP, GeckoView and Play requirements can change after this September 17, 2026 baseline.

46 45. Appendix B - Do Not Ship Checklist

- No static fake proxy pool.
- No fake developer data.
- No profile label pretending to mean isolation.
- No plaintext secrets.
- No stale engine hidden behind a current version string.
- No unbounded health-check tasks.
- No main-thread network calls.
- No hidden cross-profile access.
- No “undetectable” guarantee.
- No API 37 release without Android 17 validation.
- No browser-engine release without security review.
- No unresolved basic-navigation crash.
- No settings that only change visuals while doing nothing.
- No release with unknown signing or dependency provenance.

47 46. Final Product Identity

Kairox should be launched as:

Kairox Browser - the mobile browser built for people who need more than browsing.

It is not merely a browser with extra menus. The product architecture combines:

Browser

- + Profile Vault
- + Privacy Center
- + Network Lab
- + Developer Lab
- + Research Tools
- + Optional Local AI
- + Controlled Automation

The final experience should let a normal user stay in the simple browsing lane while a developer can open one advanced panel and immediately work with real browser internals, profile isolation, network diagnostics and debugging tools.

The engineering objective is not to make the app look complicated. The objective is to make the underlying system capable enough that the interface can stay simple.